# Structure de données et algorithmes

Projet 1: auto-complétion

27 février 2026

L'objectif du projet est d'implémenter, d'analyser théoriquement et de comparer empiriquement différentes solutions algorithmiques pour le problème de complétion automatique[1]. L'implémentation de ces différents algorithmes vous permettra de mettre en oeuvre des algorithmes de tri et une structure de données vue au cours (la file à priorité).

## Auto-complétion

Dans beaucoup d'applications ou sites web où une entrée textuelle est demandée à l'utilisateur, une liste de suggestions est souvent affichée pour compléter l'entrée courante et cette sélection évolue dynamiquement en fonction de l'entrée. Pour être utile, il est nécessaire que cette liste de suggestions soit la plus pertinente possible et s'affiche également rapidement, idéalement aussi vite que l'utilisateur tape son texte au clavier.

**Problème algorithmique.** Plus formellement, le problème est défini de la manière suivante. On suppose avoir accès à une liste de $N$ termes (des chaînes de caractères) qui représentent la liste des entrées possibles de l'utilisateur. A chaque terme, on associe un poids, une valeur entière dans ce projet, qui représente la pertinence ou la popularité du terme. On appellera requête ('query" en anglais) l'entrée courante de l'utilisateur, elle aussi une chaîne de caractères. Cette requête pourra être vide si l'utilisateur n'a pas encore entré une seule lettre.

Etant donné une requête, l'objectif est de retrouver dans la liste de termes les $k$ termes de poids les plus élevés dont la requête est un préfixe et de les ordonner, pour les afficher à l'écran, par ordre de poids décroissants. On notera dans la suite $m$ le nombre de termes, parmi $N$, dont la requête est un préfixe. Si $m < k$, seuls ces $m$ termes seront renvoyés à l'utilisateur.

**Solutions proposées.** On propose d'étudier et de comparer trois solutions à ce problème:

- **Recherche linéaire:** cette approche consiste à parcourir la liste des termes par ordre décroissant de poids et à sélectionner parmi ceux-ci les $k$ premiers dont la requête est un préfixe.
- **Tri et recherche binaire:** on trie initialement la liste des termes par ordre lexicographique. On détermine ensuite les $m$ termes dont la requête est un préfixe par une recherche binaire (ces termes sont consécutifs dans l'ordre lexicographique), on trie ces $m$ termes par ordre décroissant de poids et on extrait les $k$ premiers termes dans l'ordre obtenu.
- **File à priorité:** Comme pour l'approche précédente, on trie les termes par ordre lexicographique et on détermine les $m$ termes dont la requête est un préfixe par une recherche

[1] Ce projet s'inspire de ce projet proposé à l'université de Princeton.

| Query: `r` | Query: `ryan` |
|---|---|
| robert downey jr. | ryan reynolds (i) |
| robin williams (i) | ryan van de kamp buchanan |
| robbie coltrane | ryan hurst (i) |
| ralph fiennes | ryan seacrest |
| robert de niro | ryan c. bogdewic (ii) |
| rupert grint | ryan gosling |
| robin atkin downes | ryan phillippe |
| richard griffiths (i) | ryan preimesberger |
| richard jenkins (i) | ryan fischman |
| reggie lee (i) | ryan clark (iii) |

Figure 1: Illustration de l'auto-complétion avec le fichier `actors_2500K.txt` fourni contenant une liste de noms d'acteurs avec comme poids mesurant leur popularité. A droite (resp. à gauche) l'utilisateur a tapé `r` (respectivement `ryan`) et l'auto-complétion lui a présenté les $k$ noms d'acteurs les plus populaires qui commence par ces caractères.

binaire. Ensuite, plutôt que de trier l'entièreté des $m$ termes, on utilise une file à priorité (de taille limitée à $k$, en utilisant l'opposé du poids comme valeur de priorité) pour extraire les $k$ termes de poids les plus élevés parmi ces $m$ termes. L'idée est de parcourir les $m$ termes en insérant un nouveau terme dans la file, soit si la file ne contient pas encore $k$ termes, soit si le poids de ce terme est supérieure au terme de poids le plus faible dans la file. Dans ce dernier cas, le nouveau terme devra remplacer le terme de poids le plus faible. La liste des $k$ termes pourra alors être obtenue (en ordre inverse) par $k$ extraction du maximum de la file.

## Implémentation

Nous vous fournissons les fichiers suivants:

- `Term.h/Term.c`: qui définissent des types de données simples pour représenter un terme (`Term`) et un tableau de termes (`TermArray`). Ces deux types sont non opaques et définis complètement dans `Term.h`.
- `AutoComplete.h/AutoCompleteLinear.c`: l'implémentation de la solution par recherche linéaire.
- `main.c`: un fichier principal qui permet de lire un fichier `txt` contenant une liste de termes et leurs poids (supposés triés par ordre décroissant de poids) et ensuite soit de lancer un mode interactif permettant de tester l'auto-complétion dans un terminal, soit de lancer l'auto-complétion sur une requête donnée en ligne de commande.
- `Makefile`: permettant de compiler 5 exécutables: `autocompletelinear` correspondant à la première solution, `autocompletesort_ms` et `autocompletesort_qs` correspondant à la seconde solution en utilisant respectivement le tri par fusion et le tri rapide, `autocompletepq_ms` et `autocompletepq_qs` correspondant à la troisième solution utilisant les mêmes deux tris.
- `Data`: un répertoire contenant plusieurs fichiers `txt` contenant des listes de termes que vous pouvez utiliser pour vos tests.

Vous devez implémenter les fichiers suivants:

- `QuickSort.c` et `MergeSort.c`: implémentant une version respectivement du tri rapide et du tri par fusion utilisant l'interface du fichier d'entête `Sort.h`.

- `BinarySearch.c`: implémentant deux fonctions permettant de faire la recherche binaire nécessaire aux solutions 2 et 3. La définition précise de ces deux fonctions est données dans le fichier d'entête correspondant `BinarySearch.h`.
- `PQ.c`: implémentant une file à priorité de taille limitée en suivant l'interface du fichier `PQ.h`.
- `AutoCompleteSort.c` et `AutoCompletePQ.c`: implémentant les solutions par tri et par file à priorité de l'auto-complétion. Pour ces implémentations, vous devez obligatoirement utiliser les implémentations du tri, de la recherche binaire et de la file à priorité ci-dessus.

**Remarques.** Les implémentations de tri, de recherche binaire et de file a priorité doivent être génériques, c'est-à-dire fonctionner pour des tableaux et donnés de type `void *`. Les fonctions à définir prennent en argument les fonctions de comparaison et d'échange d'éléments nécessaires. Pour le tri par fusion, vous devrez réfléchir à une manière de l'implémenter sur base uniquement d'échanges de valeurs dans le tableau à trier. Pour y arriver, vous pouvez trier initialement un tableau d'indices vers le tableau de départ et ensuite permuter les éléments du tableau à trier sur base de ce tableau d'indices. Pour le tri rapide, vous pouvez implémenter la variante de votre choix pour le choix du pivot. Pour toutes les fonctions, vos implémentations doivent être les plus efficaces possible.

## Analyses théorique et empirique

Dans votre rapport, répondez aux question suivantes:

1. Calculez et rapportez dans une table les temps de calculs des deux algorithmes de tri sur les fichiers `actors_25k.txt`, `actors_250k.txt` et `actors_2500k.txt` fournis, lorsque vous les triez par ordre lexicographique et selon leurs poids.
2. Discutez les résultats. En particulier, comparez les temps de calcul obtenus aux complexités théoriques des deux algorithmes de tri et déterminez lequel des deux algorithmes vous semble le plus approprié pour ce problème.
3. Analysez la complexité en temps et en espace des trois approches d'auto-complétion dans le meilleur et dans le pire cas en fonction du nombre total de termes $N$, du nombre $m$ de termes dont la requête est un préfixe et du nombre $k$ de termes à afficher. Si nécessaire, faites une analyse séparée en fonction de l'algorithme de tri utilisé, parmi les deux considérés. Pour cette analyse, prenez en compte vos choix d'implémentation, en les précisant si besoin dans le rapport (par exemple, le choix du pivot pour le `QuickSort`).
4. Comparez les trois approches d'auto-complétion empiriquement. Pour cela, fixez $k$ à 20 et étudiez les performances de chaque algorithme en fonction de $l$ la longueur de la requête, pour $l$ allant par exemple de 0 à 10. Pour faire cela, sélectionnez au hasard un terme et utilisez comme requête ses $l$ premiers caractères. Répetez l'expérience un certain nombre de fois (au moins 100) et moyennez les temps de calcul ainsi obtenus pour les appels à `acComplete`. Reportez sur un graphe l'évolution des temps de calcul en fonction de $l$ pour les trois approches avec l'algorithme de tri choisi au point précédent. Répétez l'expérience pour les trois fichiers `actors_25k.txt`, `actors_250k.txt` et `actors_2500k.txt` pour étudier également l'impact de $N$. Pour ces expériences, vous pouvez vous contenter d'utiliser seulement l'algorithme de tri sélectionné au point 2.
5. Discutez les résultats obtenus au point précédent. En particulier, comparez les trois approches entre elles, discutez l'impact de $l$ et de $N$ sur chaque approche et mettez les résultats empiriques en perspective par rapport aux complexités théoriques.
6. Concluez quant à l'approche qui vous semble la meilleure.

## Date de remise et soumission

Le projet est à réaliser ***seul ou en binôme*** pour le ***dimanche 22 mars 2026 à 23h59*** au plus tard. Le projet est à remettre via Gradescope (http://gradescope.com, code cours **YBZ7RR**).

Les fichiers suivants doivent être remis (dans deux projets séparés sur Gradescope, pour le rapport et le code):

1. Votre rapport au format PDF et nommé `rapport.pdf`. Soyez bref mais précis et respectez bien la numération des questions;
2. Les fichiers `MergeSort.c` et `QuickSort.c`.
3. L'implémentation de la file à priorité `PQ.c`.
4. Les deux solutions `AutoCompleteSort.c` et `AutoCompletePQ.c`
5. Le fichier `declaration.txt` rempli, où vous indiquerez les contributions des différents membres du groupe et certifierez avoir respecté les règles du cours en matière de plagiat et d'utilisation d'outils d'IA consultables sur Ecampus.

Respectez bien les extensions de fichiers ainsi que les noms des fichiers, en ce compris les minuscules/majuscules. N'incluez aucun fichier supplémentaire.

Un projet non rendu à temps recevra une cote globale nulle. La soumission du projet suppose que vous avez pris connaissance des règles en terme de plagiat rappelées sur Ecampus. En cas de plagiat[2] avéré, le groupe se verra affecter une cote nulle à l'ensemble du cours.

Les critères de correction sont précisés sur Ecampus.

**Bon travail !**

[2]Des tests anti-plagiat seront réalisés.